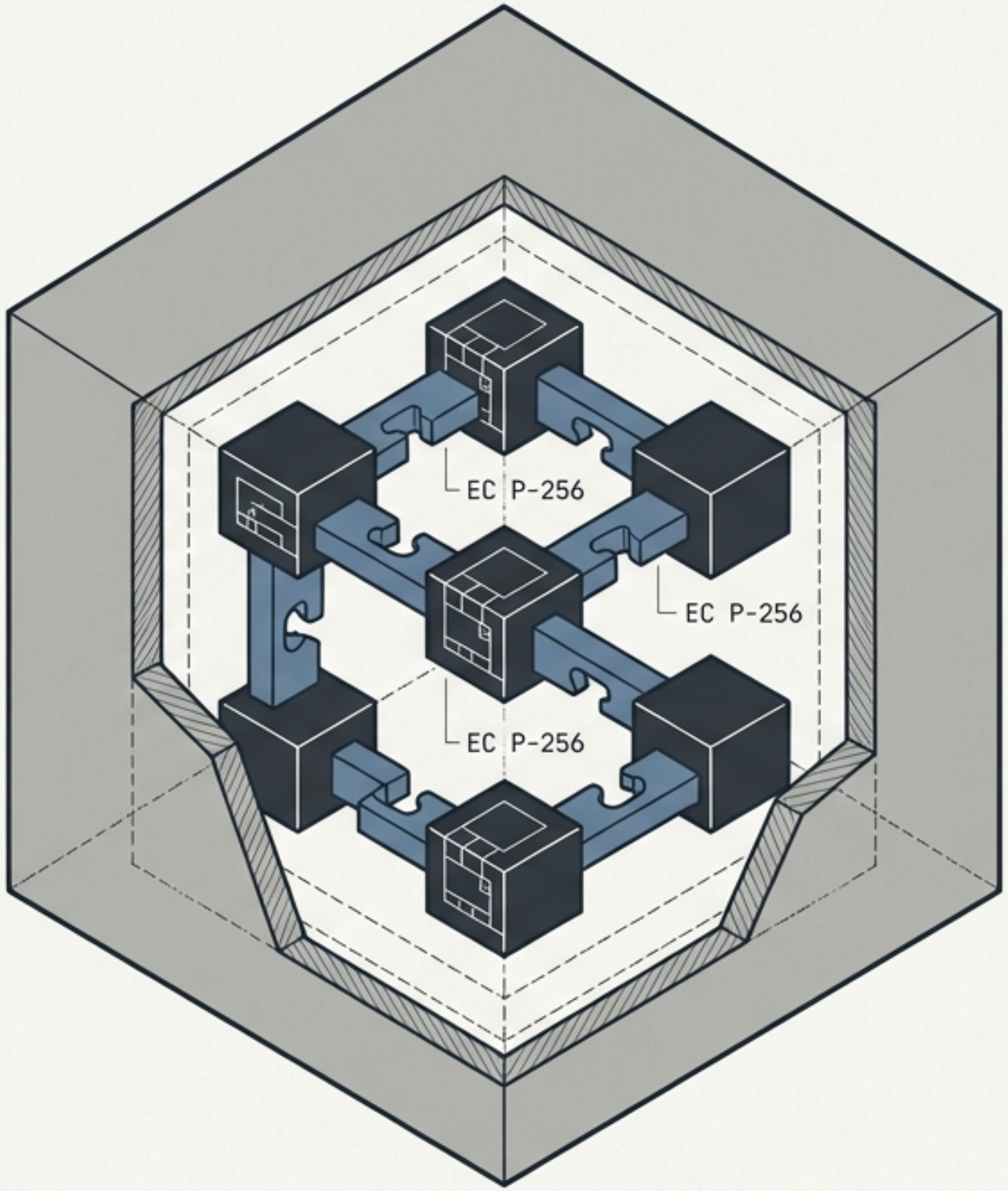
Version Control with Integrated Trust

Standard version control delegates security to a central server, relying on policies, passwords, and plaintext.

SG-Vault replaces server-side policy with cryptographic mathematics. By combining zero-knowledge storage with a PKI-native graph architecture, it unlocks secure automation, trustless collaboration, and unforgeable provenance.



Shifting from Policy to Cryptography

	Standard VCS	SG-Vault Architecture
Server Visibility	**Plaintext.** Server sees all code, filenames, and history.	**Zero-Knowledge.** Server sees only opaque, encrypted blobs.
Identity & Provenance	**Email strings & Server accounts.** Easily forged locally.	**EC P-256 Keypairs.** Every commit is a mathematically unforgeable signature.
Conflict Location	**Shared Remote.** Conflicts block team progress.	**Local Clone.** Conflicts are strictly quarantined to the local device.
Automation Auth	**Omnipotent API Keys.** High risk of credential theft.	**Write-Only Change Packs.** Cryptographically scoped, append-only payloads.

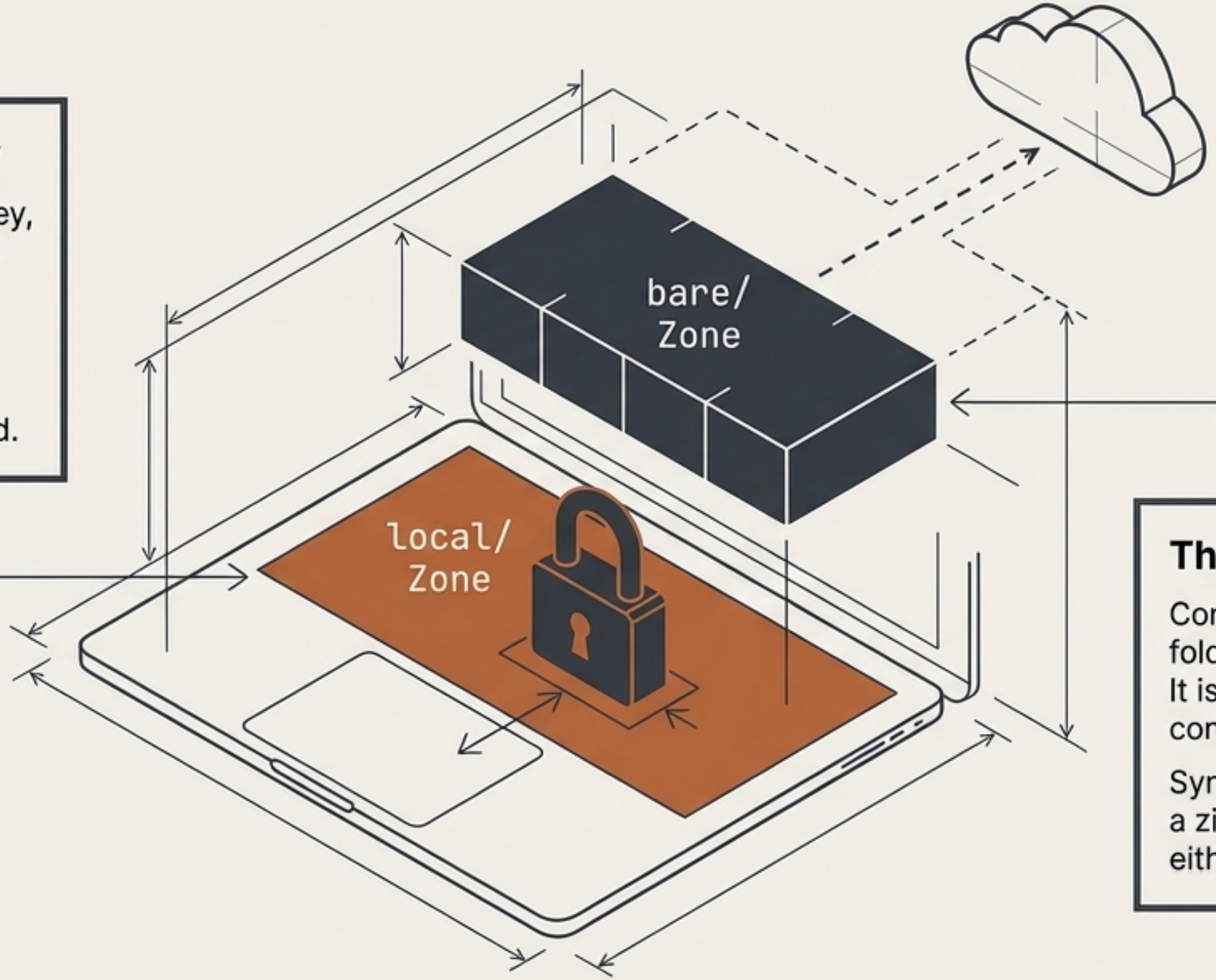
The Storage Boundary: Segregating Identity from Data

Device-Bound Identity

Contains the Clone Private Key, Vault Key, and tracking state.

It never leaves the specific device and is never synced.

Provenance cannot be cloned.

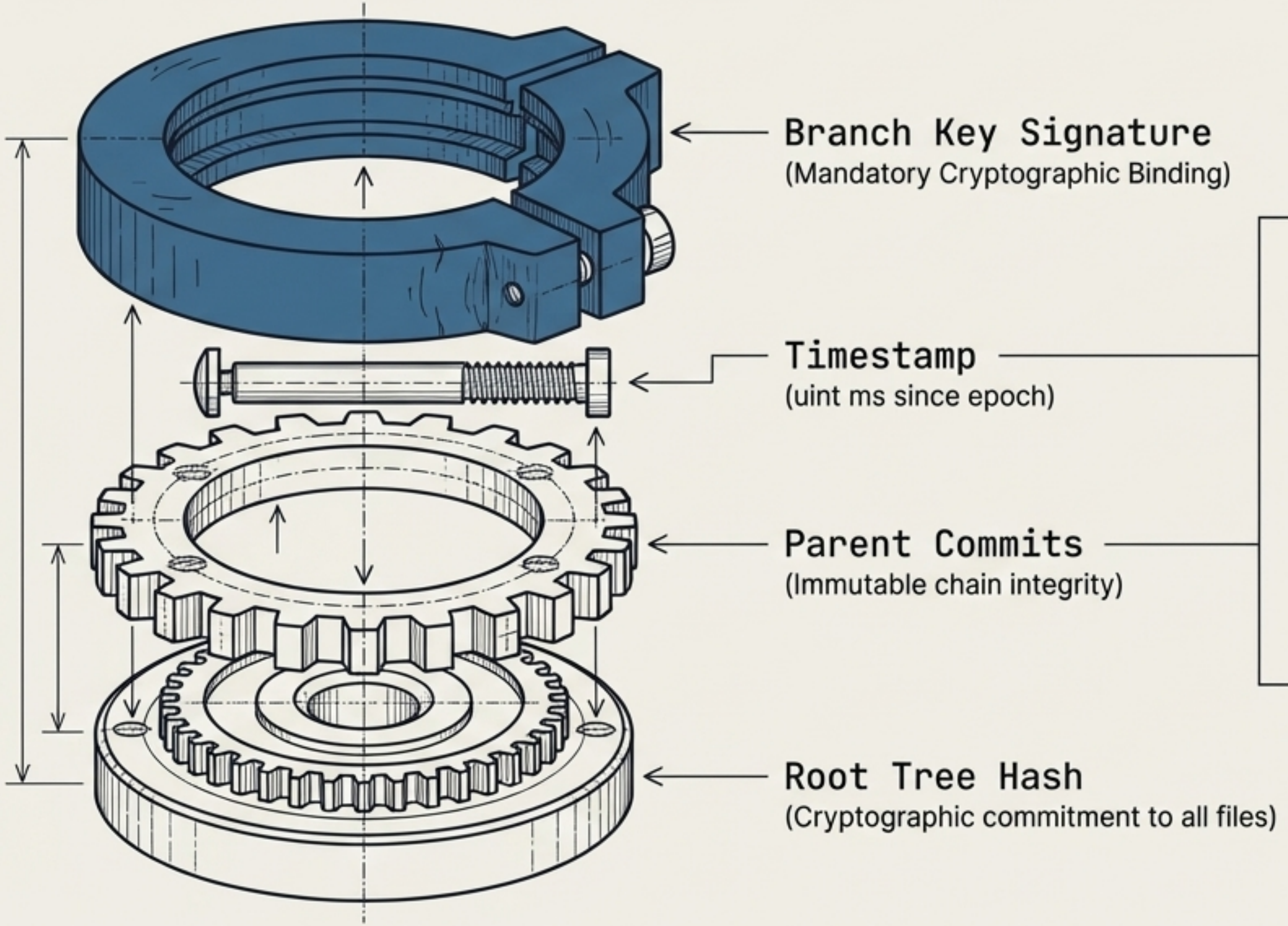


The Portable Vault

Contains opaque, encrypted folders (data/, refs/, indexes/). It is the complete, self-contained vault.

Syncs to the server, packs into a zip, and survives deletion of either side.

Provenance is in the Graph, Not the Policy



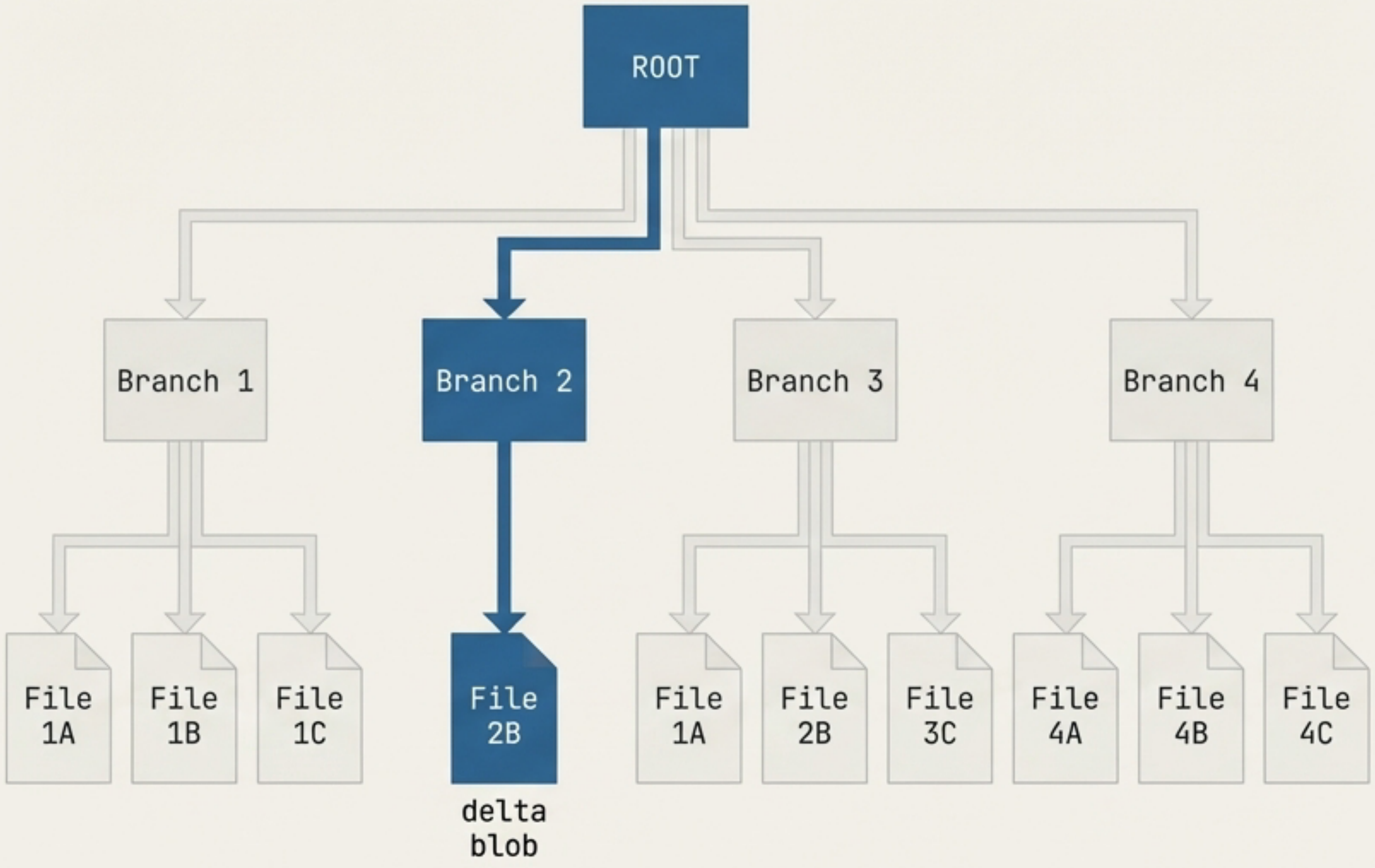
The Interlocking Guarantee

Tampering with any internal field changes the resulting commit ID. Tampering with any historical commit mathematically invalidates all descendant commit IDs.

Mandatory Cryptography

The outer Branch Key Signature is mandatory on creation. There are no unsigned commits. The server does not enforce this via policy; the cryptographic structure simply demands it.

The Sub-Tree Shortcut: Fast Sync Without Diffs



Natural Delta Pushes

Change one file, and only the changed blob and its direct upward path of tree nodes are new. Everything else is already on the server. No diff algorithm is needed.

Cheap, Blind Merges

At each tree node, the system compares encrypted `tree_id` values [redacted]
If a sub-tree ID matches, the sub-folder is identical and safely skipped without decryption.

Automatic Deduplication

Identical sub-trees across different branches share the same content-addressed objects. One encrypted copy is securely referenced by many commits.

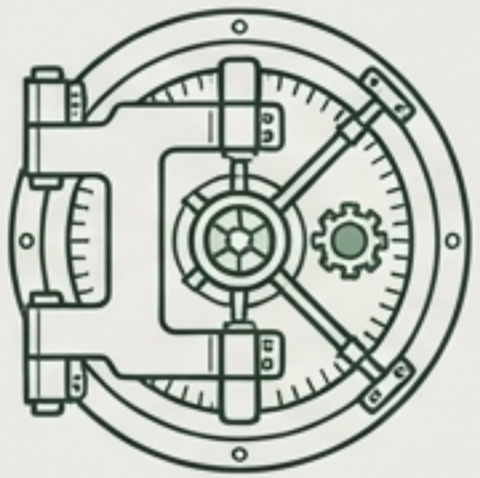
Cryptographic Identity: The End of Shared Credentials

Clone Branch Key
(Provenance)

`Scope:`	Auto-generated on clone. Strictly local to one specific device and session.
`Function:`	Proves exact device provenance. If the local key is lost, the branch is permanently locked (a standard lifecycle event).

User Main Key
(Recovery)

`Scope:`	Multi-device cryptographic identity.
`Function:`	Stored securely in the OS keychain or a secure password vault. Used exclusively to re-import identities across devices.

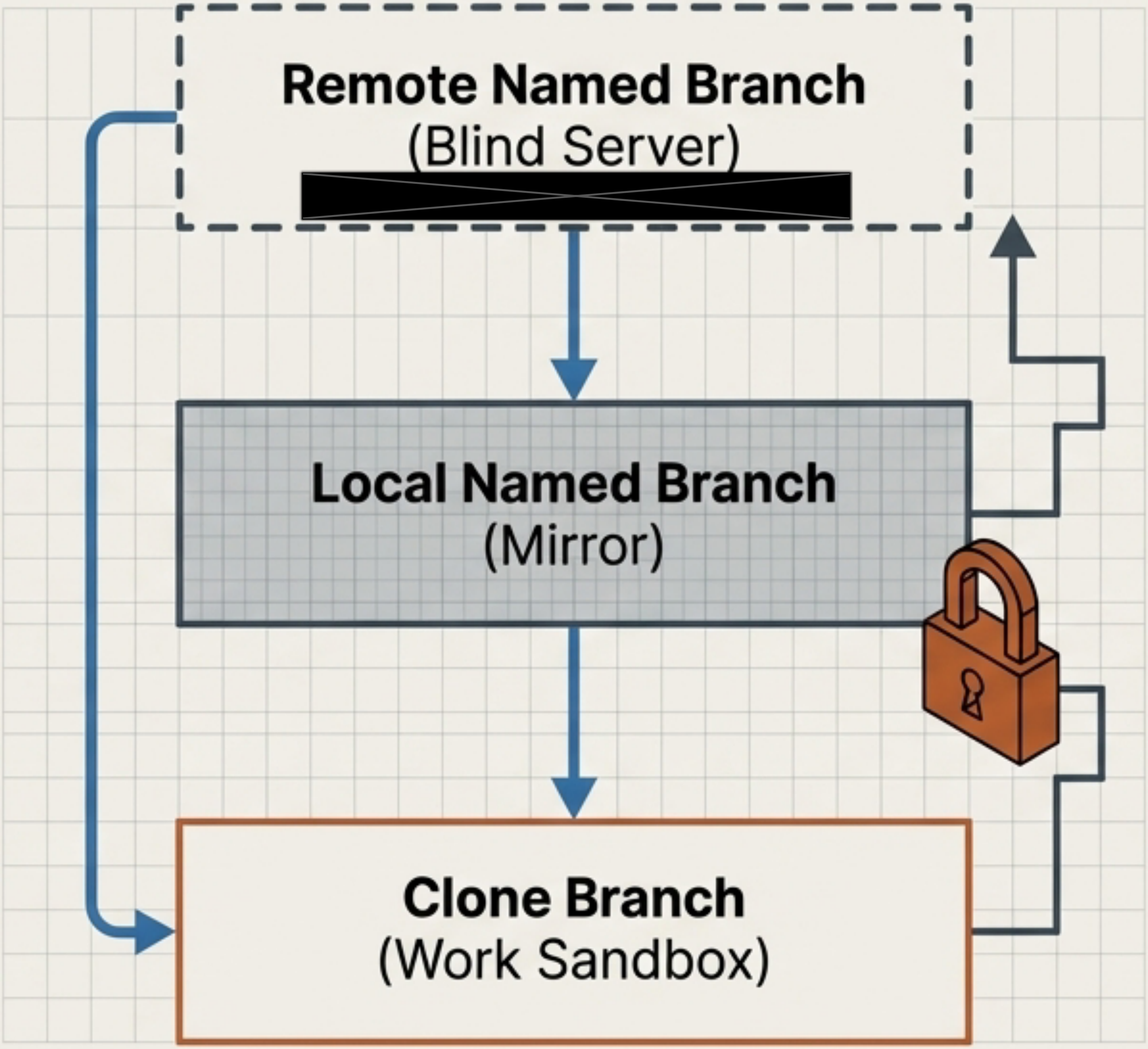


Named Branch Key
(Authority)

`Scope:`	Lives encrypted directly inside the bare vault data.
`Function:`	The authoritative integration point. Decrypted via the vault `read_key` to authorize team-wide merges. Direct commits are strictly rejected.

The Three-Layer Gravity Model

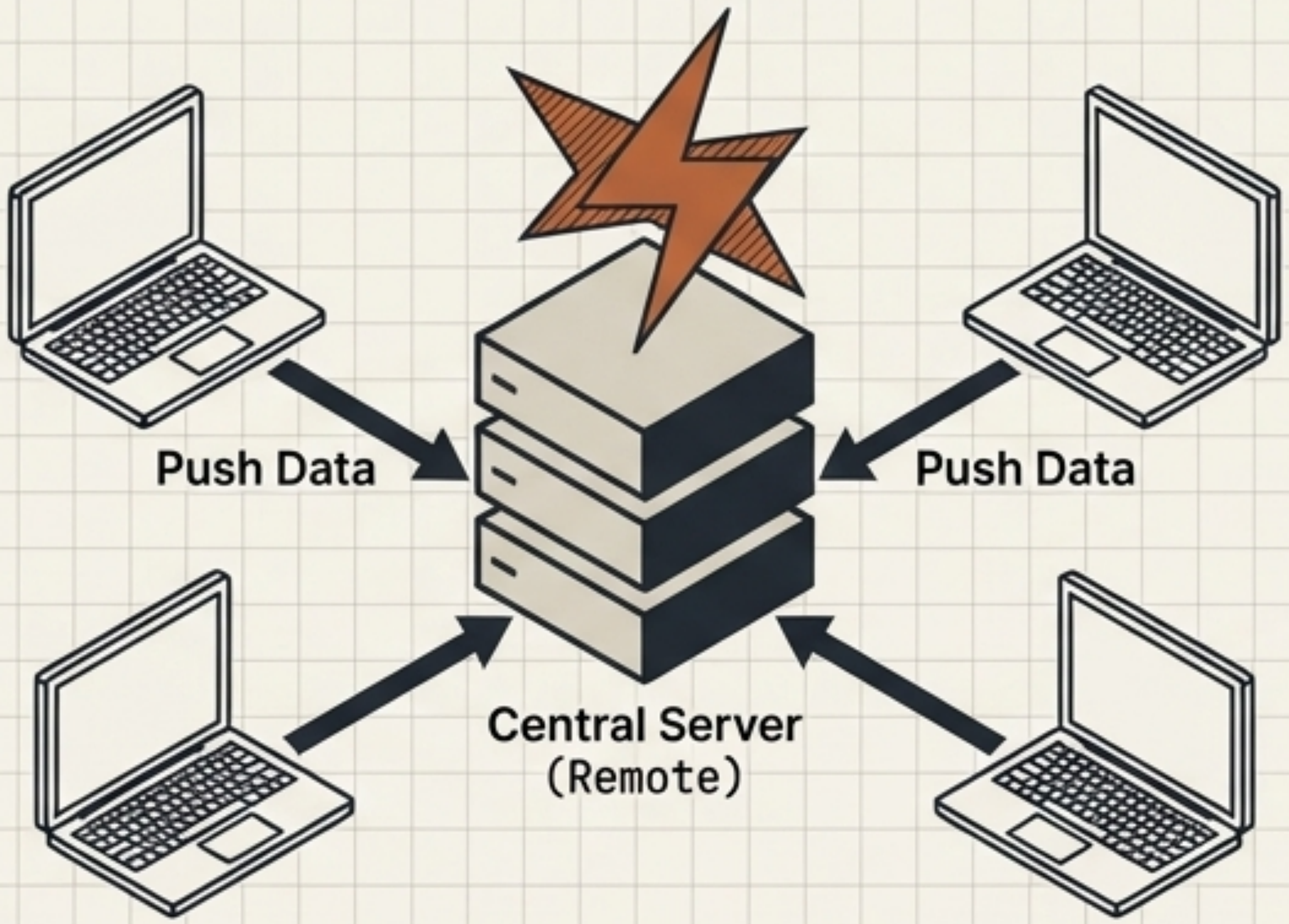
Falling Down (Fetch/Pull).
The Local Named branch accepts Remote changes via pure Fast-Forward. It simply moves the pointer to match the authority effortlessly.



Lifting Up (Merge/Push).
[redacted]
Pushing upward requires cryptographic effort. The Clone Branch must initiate a signed Merge Commit (authorized by the named branch key) before pushing encrypted deltas back up.

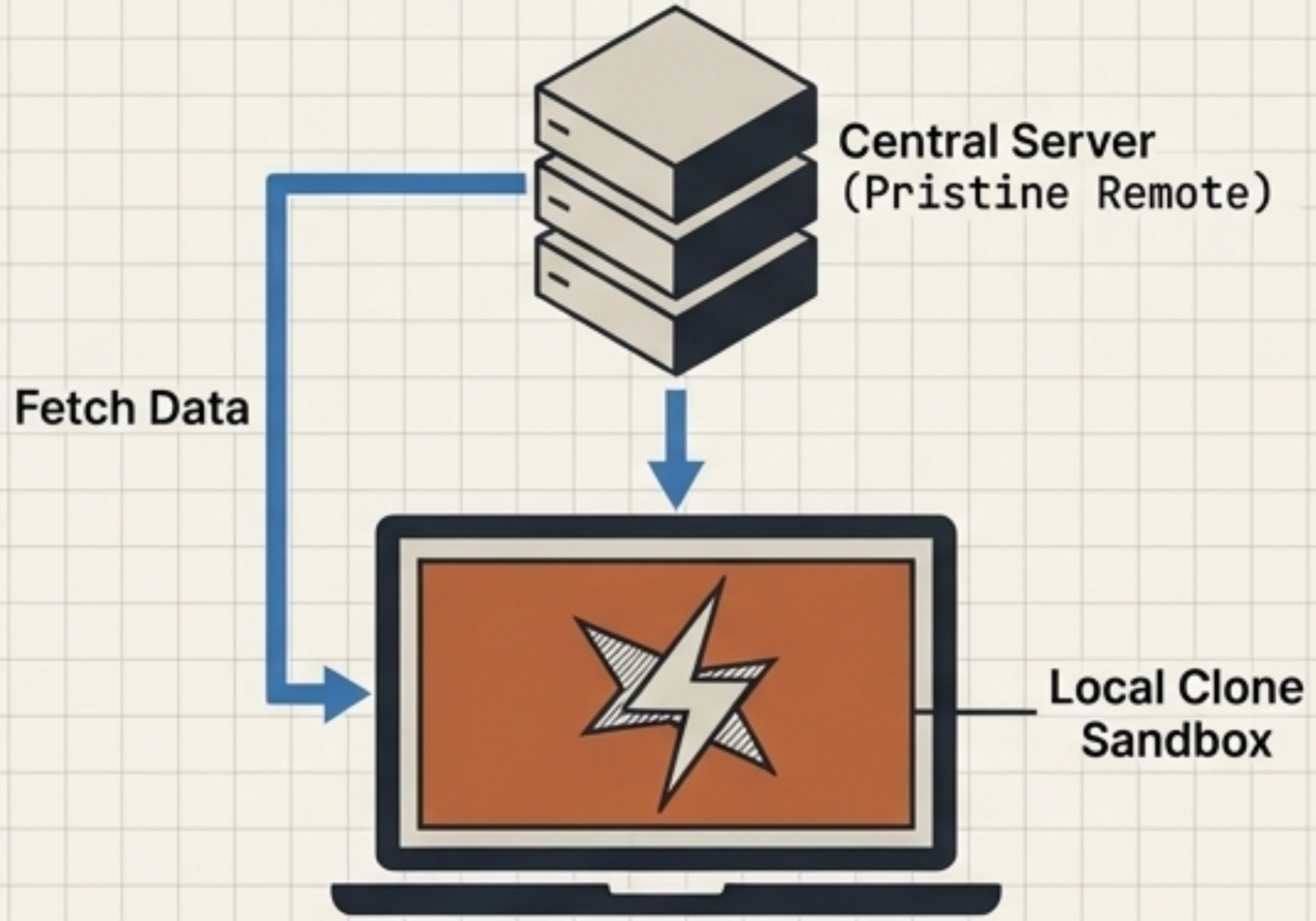
Conflict Quarantine: The Remote is Always Pristine

The Standard Model



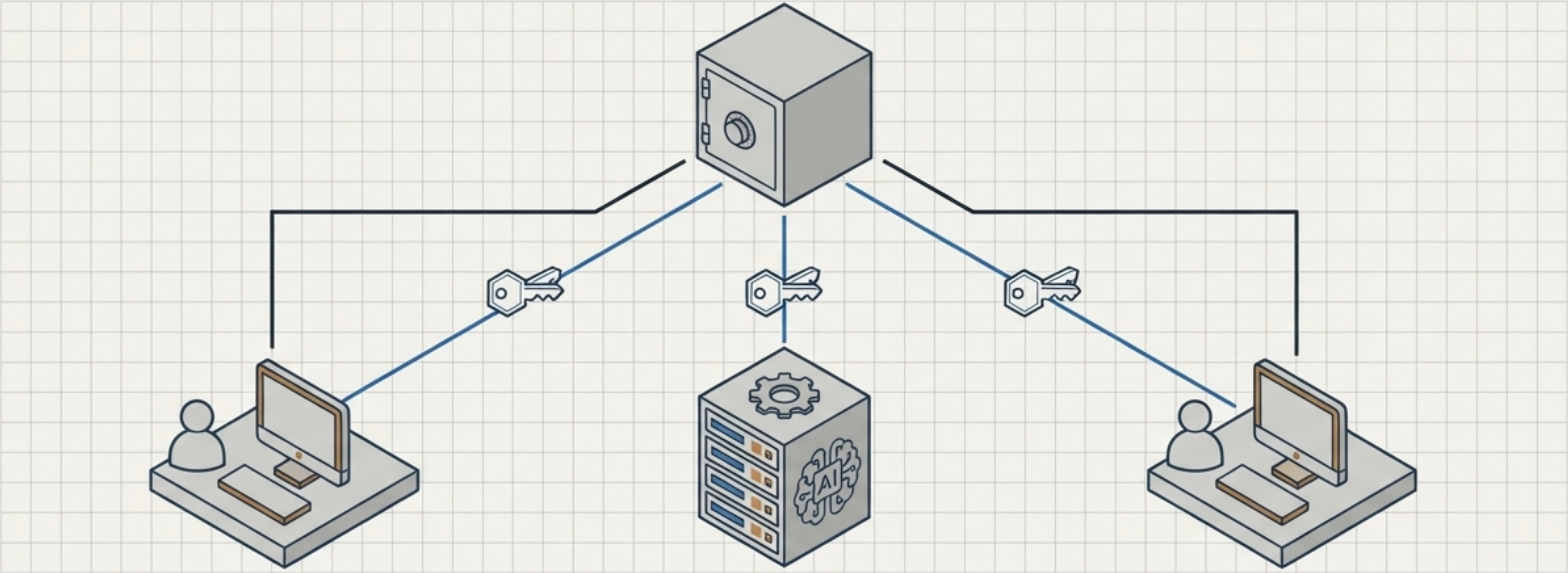
Pushing to a shared remote often triggers server-side conflicts. The remote breaks, blocking the entire team until someone resolves the conflict remotely.

The Quarantine Model



Conflicts never exist on the named remote branch. Push operations mandate a local fetch first. Divergence is pulled down and absorbed exclusively by the local device.

The `.conflict` Convention: Conflicting files adopt the remote authority by default. Local changes are preserved locally as `README.conflict.md`. Pushing is mathematically blocked until the local user resolves the quarantine.



No Special API Keys Needed

Because identity is tied to an auto-generated clone branch key rather than an email or password, AI agents receive the exact same first-class identity as human engineers. The architecture makes no distinction.

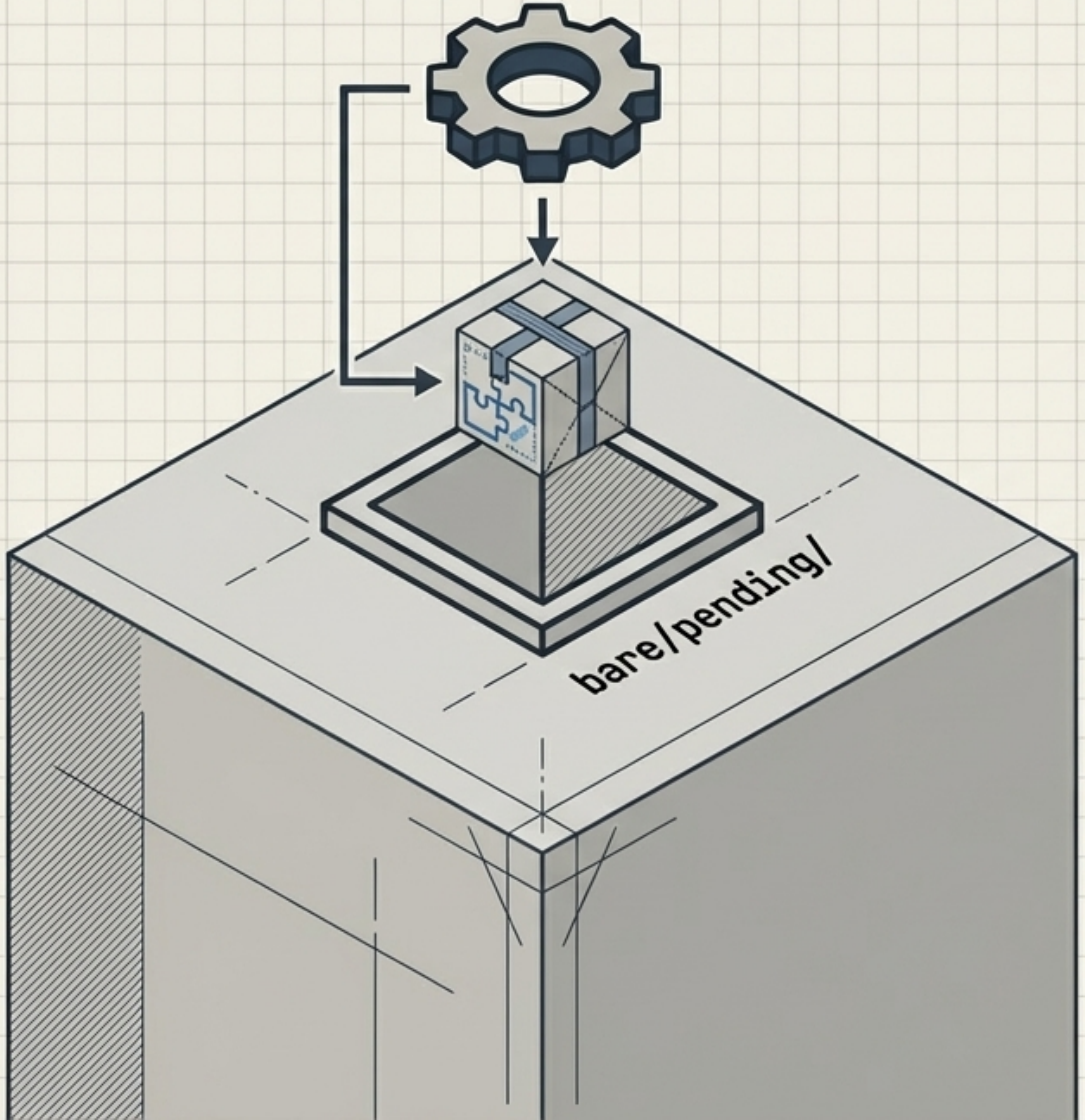
Cryptographic Accountability

Every single action taken by an autonomous agent is mathematically signed by its unique, session-specific branch key, ensuring perfect auditability.

Perfect Isolation

Agents work in their own cryptographically isolated clone branches. They cannot overwrite the named branch; they can only submit signed merges for human (or quorum) review.

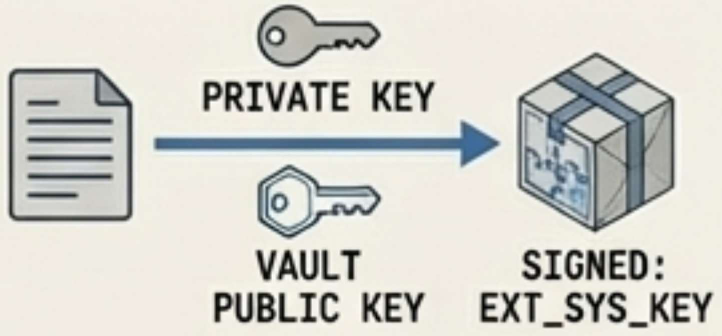
Change Packs: High-Frequency, Keyless Telemetry



How do external systems write to a zero-knowledge vault if they lack the read key or the time for a full clone? They use append-only Change Packs.

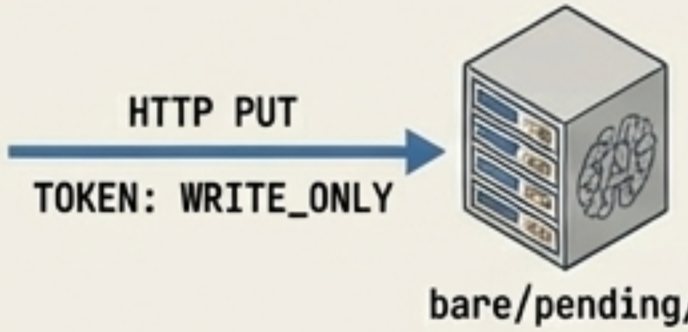
1. Package & Sign

The external system packages a data blob and signs it with its own key, utilizing the vault's public key shared out-of-band.



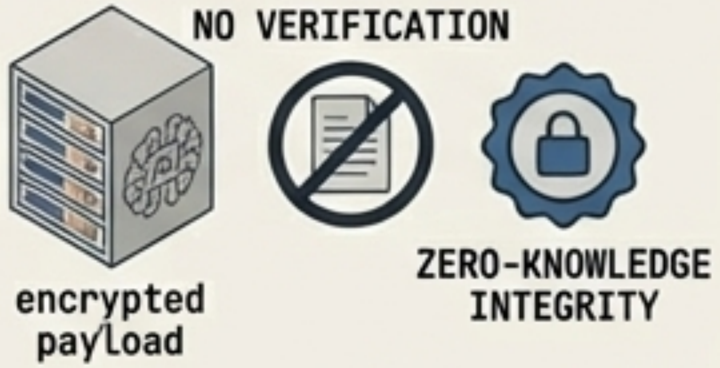
2. The Single PUT

Using a strictly scoped, write-only token, the external system executes a single HTTP PUT request directly into the bare/pending/ namespace.

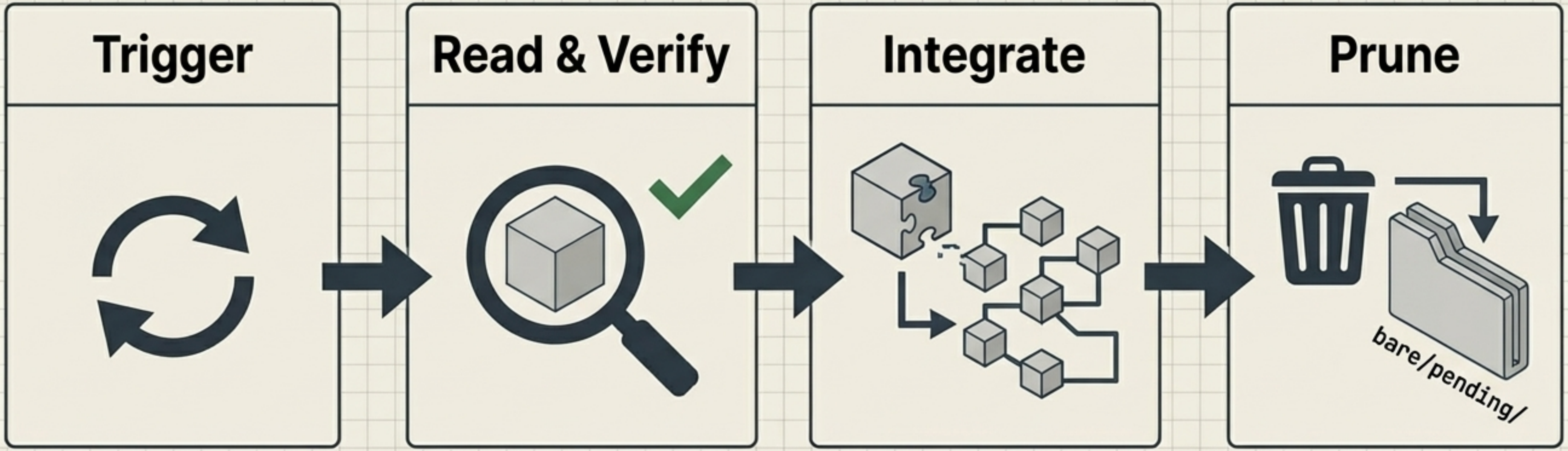


3. The Dumb Server

The server accepts the encrypted payload blindly. It does not verify the signature or read the contents, maintaining absolute zero-knowledge integrity.



The GC Drain: Verifying the Unseen



Triggered automatically on any standard sync (Push, Pull, or Merge) by a client possessing the vault read key.

The client reaches into bare/pending/, decrypts the payload locally, and verifies the internal cryptographic signature.

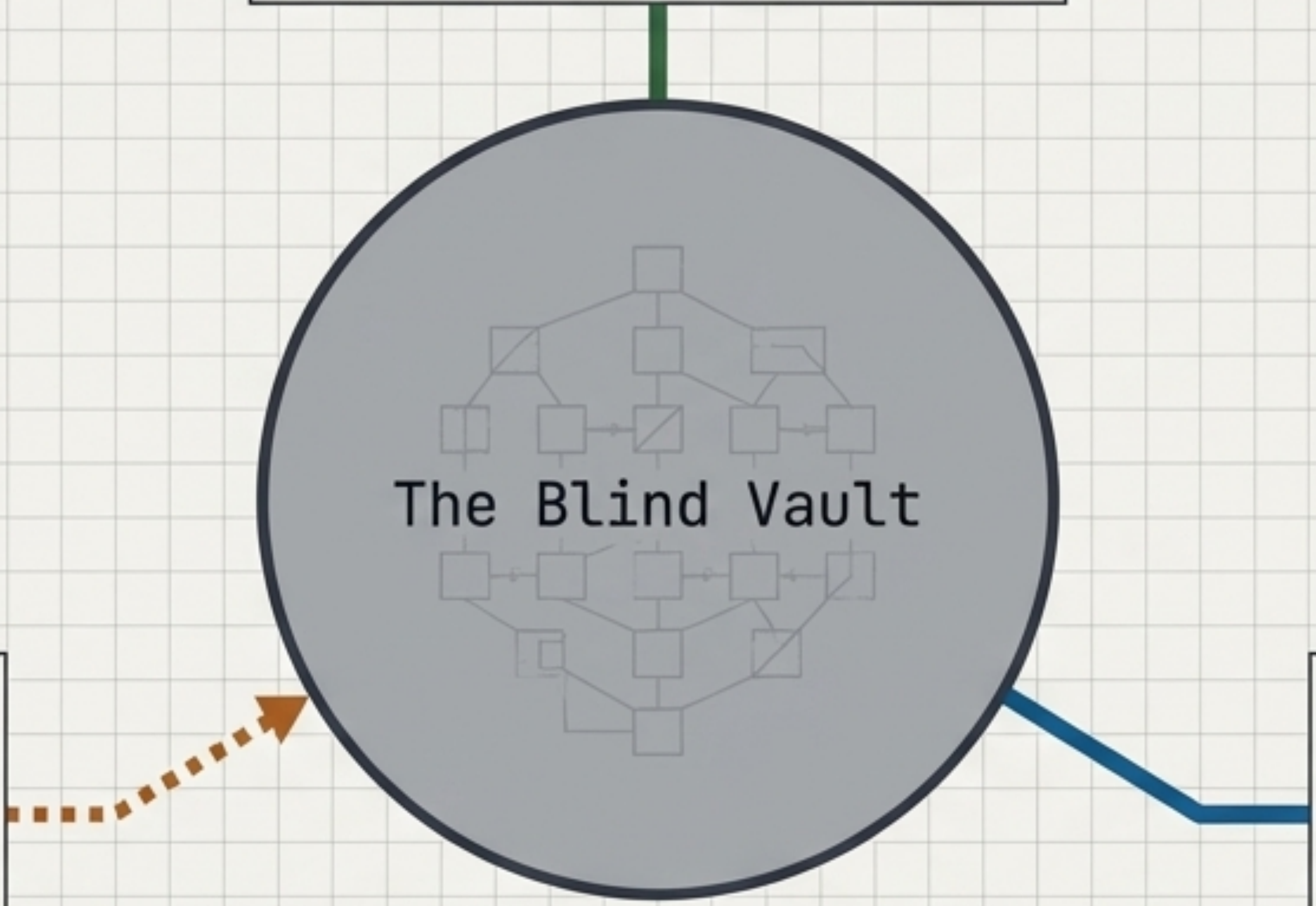
If valid, the blob is woven directly into the graph as a true commit, securing its provenance.

Only after a successful health check and reference update is the pending object pruned. The client enforces the truth.

A Unified Ecosystem of Trust

The Human

Read/Write Authority. Holds the vault key. Pulls history, resolves conflicts locally, and pushes unforgeable signed commits.



The External System

Append-Only. Holds no read keys. Uses a scoped token to drop signed, encrypted telemetry into bare/pending/ via a single HTTP request.

The AI Agent

First-Class Identity. Auto-generates a clone key. Works in perfect cryptographic isolation and submits signed merges for review.

One identical underlying graph architecture securely authenticates the human, empowers the AI, and safely ingests telemetry—all without the server ever seeing a single plaintext byte.